

УО «Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра ПОИТ

Отчет по лабораторной работе №2

по предмету

Базы данных

Вариант 1

Выполнил:

Бражалович А.И.

Проверил:

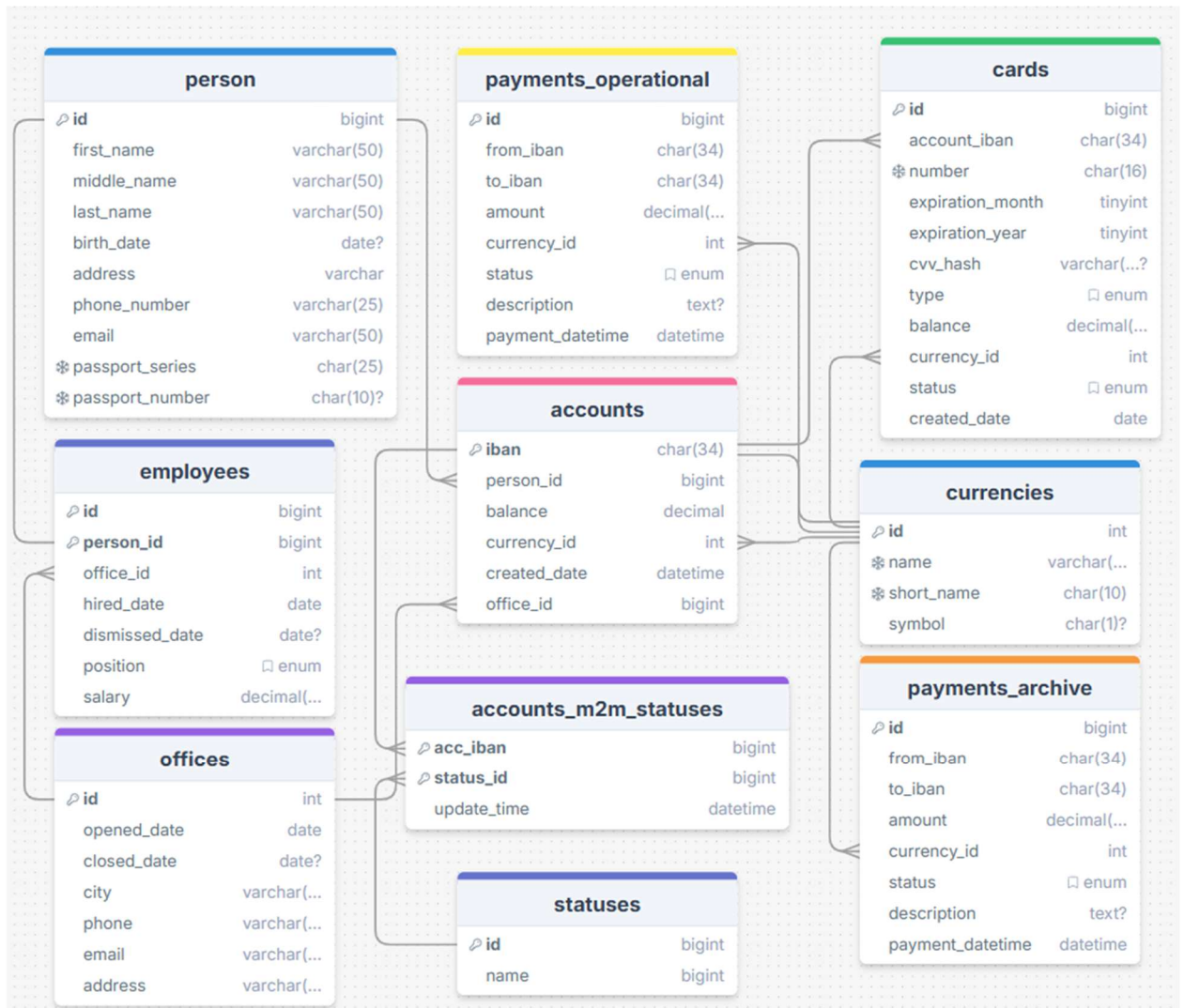
Фадеева Е.Е.

Группа 351004

Минск 2025

Новая модель

Начальная модель:



1. Какие каскадные операции необходимы в этой базе данных?

Родительская Таблица	Дочерняя Таблица	Действие ON DELETE	Действие ON UPDATE
person (id)	accounts	CASCADE	CASCADE
person (id)	employees	CASCADE	CASCADE
offices (id)	employees	SET NULL	CASCADE
offices (id)	accounts	SET NULL	CASCADE
accounts (iban)	cards	CASCADE	CASCADE
accounts (iban)	accounts_m2m_statuses	CASCADE	CASCADE

Родительская Таблица	Дочерняя Таблица	Действие ON DELETE	Действие ON UPDATE
accounts (iban)	payments_operational (from_iban, to_iban)	RESTRICT	CASCADE
accounts (iban)	payments_archive (from_iban, to_iban)	RESTRICT	CASCADE
currencies (id)	Все	RESTRICT	CASCADE

2. Существует ли возможность аномалий операций вставки, обновления, удаления данных?

accounts

При создании аккаунта поле created_date будет дублировать дату в таблице account_statuses_to_accounts при комбинации id аккаунта и статуса «создан».

accounts_m2m_statuses

Нельзя записать новый статус, пока не существует счет (acc_iban), хотя таблица статусов (statuses) существует.

3. Можно ли использовать схемы «звезда» или «снежинка» с этой базой данных, чтобы избежать некоторых аномалий операций с данными?

Схема «Звезда» не устраняет аномалии операций с данными в транзакционном смысле. Напротив, она вносит избыточность (аномалии обновления) для повышения производительности аналитики.

4. Составьте список всех функциональных зависимостей в базе данных.

Таблица	Первичный ключ (Ключ)	Функциональная зависимость (ФЗ)
person	id	last_name, first_name, middle_name, birth_date, phone_number, registration_address
offices	id	opened_date, closed_date, city, phone, email, address
employees	id	person_id, office_id, hired_date, dismissed_date, position, salary

	person_id	id, office_id, hired_date, dismissed_date, position, salary
currencies	id	name, short_name, symbol
	name	id, short_name, symbol
	short_name	id, name, symbol
accounts	iban	person_id, office_id, balance, currency_id, created_date
statuses	id	name
cards	id	account_iban, number, expiration_month, expiration_year, cvv_hash, type, balance, currency_id, status, created_date
	number	id, account_iban, expiration_month, expiration_year, cvv_hash, type, balance, currency_id, status, created_date
payments_operational	id	from_iban, to_iban, amount, currency_id, status, comment, payment_datetime
payments_archive	id	from_iban, to_iban, amount, currency_id, status, comment, payment_datetime

5. Существуют ли отношения, имеющие многозначные зависимости? Если «да», как можно переработать схему, чтобы избежать таких зависимостей?

Нет отношений, имеющих многозначные зависимости.

6. Нарушает ли схема какие-либо «требования нормализации»? Если «да», доработайте схему, чтобы избежать таких нарушений.

Не нарушает.

7. Существуют ли какие-либо потенциальные проблемы с производительностью базы данных? Если «да», запишите их.

1. Избыточные и неиспользуемые индексы (ключи)

- Таблица employees: Атрибуты id и person_id оба являются первичными ключами (так как person_id является UNIQUE и ссылается на

person(id)). Наличие двух уникальных индексов (один из которых является кластерным РК, а другой — вторичным UNIQUE-индексом) на одной таблице может немного увеличить накладные расходы при вставке/обновлении данных и потребовать больше места для хранения индексов.

- Таблица cards: Аналогично, id и number оба являются уникальными ключами. Если number является тем атрибутом, по которому чаще всего ищут карту, то id может быть избыточным как РК, если только он не нужен для ссылок из других таблиц (чего здесь нет).

2. Типы данных и широкие ключи

- Широкие ключи в accounts и payments: IBAN (CHAR(34)) используется как первичный ключ в accounts и как внешний ключ во многих других таблицах (cards, payments_operational, payments_archive, accounts_m2m_statuses). Использование широких строковых ключей (34 символа) вместо более узких целочисленных ключей может значительно снизить производительность, так как:
 - Увеличенное хранение: Индексы, использующие IBAN, будут намного больше, чем индексы на BIGINT.
 - Медленный поиск/сравнение: Сравнение строк занимает больше времени, чем сравнение чисел.
 - Большие внешние ключи: Все таблицы, ссылающиеся на accounts (включая accounts_m2m_statuses с его составным ключом), несут накладные расходы на хранение 34 символов в каждом внешнем ключе.
- Чрезмерное использование BIGINT: Использование BIGINT (8 байт) для id в person и employees может быть избыточным, если ожидаемое количество записей не превысит 2^{31} (лимит INT - 4 байта). Более узкие типы данных экономят место в таблице и индексах.

3. Составной ключ в accounts_m2m_statuses

- Широкий составной ключ: Первичный ключ account_iban, status_id, очень широк из-за account_iban (CHAR(34)). Это утяжеляет поиск и увеличивает размер индекса, особенно при большом объеме данных истории статусов.

4. Потенциальные накладные расходы с ENUM

- Использование ENUM: В MySQL ENUM может быть эффективным для хранения, но его использование во внешних ключах или при изменении списка разрешенных значений может вызвать проблемы.

Изменение списка разрешенных позиций в employees.position требует изменения схемы. Для крупномасштабного приложения

часто лучше использовать справочную таблицу (positions) и ссылаться на нее по целочисленному FK.

8. Для каждого отношения в базе данных определите, в какой нормальной форме оно находится.

employees: BCNF

offices: BCNF

payments_operational

payments_archive: BCNF

currencies: BCNF

person: BCNF

cards: BCNF

statuses: BCNF

accounts_m2m_statuses: BCNF

accounts: BCNF

9. Есть ли отношения с возможной, но ненужной дальнейшей нормализацией? Составьте список.

Нет таких отношений

10. Можно ли добиться некоторого повышения производительности за счёт денормализации схемы? Обоснуйте своё мнение.

Область денормализации	Избыточность	Повышение производительности
accounts	Дублировать first_name, last_name клиента из person	Ускоряются запросы, которые ищут счета по имени клиента.
cards	Дублировать balance счета (accounts.balance)	Ускоряет получение текущего баланса без JOIN к таблице accounts.
Платежи	В payments_operational и payments_archive добавить currency_short_name	Ускоряет чтение информации о платеже с его валютой без JOIN к currencies.

11. Можно ли добиться некоторого повышения производительности, добавив в схему кэширующие отношения? Обоснуйте своё мнение.

Вместо частого соединения `cards → accounts → person`, можно создать кэширующую таблицу `card_owner_cache` с `card_id`, `owner_name`, `owner_phone`.

12. Добавьте в базу данных все необходимые индексы, представления, хранимые процедуры и т. д.

Индексы (person):

- `idx_perosn_name`: На столбцы для быстрого поиска по имени.

```
CREATE INDEX idx_person_name ON person (last_name, first_name, middle_name);
```

- `idx_person_contact`: На часто используемые контактные данные.

```
CREATE UNIQUE INDEX idx_person_contact ON person (email);
```

```
CREATE INDEX idx_person_phone ON person (phone);
```

Индексы (office):

- `idx_offices_city`: Для поиска по городу.

```
CREATE INDEX idx_offices_city ON offices (city);
```

Индексы (employees):

- `idx_employees_position`: Для поиска по должности.

```
CREATE INDEX idx_employees_position ON employees (position, dismissed_date);
```

Индексы (accounts):

- `idx_accounts_office_currency`: Для запросов, связанных с агрегацией по офисам и валютам (FK уже есть, но составной может быть полезен).

```
CREATE INDEX idx_accounts_office_currency ON accounts (office_id, currency_id, balance);
```

Индексы (accounts_m2m_statuses):

- `idx_accounts_status_history`: Для быстрого поиска истории по времени.

```
CREATE INDEX idx_accounts_status_history ON accounts_m2m_statuses  
(account_iban, update_datetime DESC);
```

Индексы (cards):

- idx_cards_account_status: Для быстрого поиска активных карт по счету.

```
CREATE INDEX idx_cards_account_status ON cards (account_iban, status,  
expiration_year, expiration_month);
```

Индексы (payments_operational и payments_archive):

- idx_payments_datetime: Для поиска по дате/времени (критически важно для платежей).

```
CREATE INDEX idx_payments_datetime ON payments_operational  
(payment_datetime DESC);
```

```
CREATE INDEX idx_archive_datetime ON payments_archive  
(payment_datetime DESC);
```

- idx_payments_status: Для фильтрации по статусу (особенно для операционных).

```
CREATE INDEX idx_payments_status ON payments_operational (status);
```

Представления v_active_employees_full_info

Отображает все данные об активных сотрудниках (не уволенных) с полной информацией о человеке и офисе.

```
CREATE VIEW v_active_employees_full_info AS  
SELECT  
e.id AS employee_id, e.position, e.hired_date, e.salary,  
p.last_name, p.first_name, p.phone, p.email AS personal_email,  
o.city AS office_city, o.address AS office_address, o.phone AS office_phone  
FROM employees e  
JOIN person p ON e.person_id = p.id  
JOIN offices o ON e.office_id = o.id  
WHERE e.dismissed_date IS NULL;
```

Представления v_client_accounts_summary

Отображает сводную информацию о текущих счетах клиента.


```

CREATE VIEW v_client_accounts_summary AS
SELECT
p.last_name, p.first_name, p.passport_number,
a.iban, a.balance, c.short_name AS currency_code,
o.city AS office_city, a.created_date
FROM accounts a
JOIN person p ON a.person_id = p.id
JOIN currencies c ON a.currency_id = c.id
JOIN offices o ON a.office_id = o.id;

```

Представления v_current_account_status

Отображает текущий (последний) статус каждого счета.

```

CREATE VIEW v_current_account_status AS
SELECT a.iban, s.name AS status_name, h.update_datetime
FROM accounts_m2m_statuses h
JOIN statuses s ON h.status_id = s.id
WHERE (h.account_iban, h.update_datetime) IN (
SELECT account_iban, MAX(update_datetime) AS max_dt
FROM accounts_m2m_statuses
GROUP BY account_iban
);

```

Процедура sp_transfer_funds

```

CREATE PROCEDURE sp_transfer_funds (
IN p_from_iban CHAR(34),
IN p_to_iban CHAR(34),
IN p_amount DECIMAL(19,4),
IN p_currency_id INT,
IN p_comment
)
BEGIN
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
-- При возникновении ошибки откатить транзакцию
ROLLBACK;
SIGNAL SQLSTATE '45000' SET MESSAGE_ = 'Transaction failed. Rolled
back.';
END;
START TRANSACTION;
-- 1. Проверка наличия средств
IF (SELECT balance FROM accounts WHERE iban = p_from_iban) < p_amount
THEN

```

```

SIGNAL SQLSTATE '45000' SET MESSAGE_ = 'Insufficient funds.';
END IF;
-- 2. Списание со счета отправителя
UPDATE accounts
SET balance = balance - p_amount
WHERE iban = p_from_iban;
-- 3. Зачисление на счет получателя
UPDATE accounts
SET balance = balance + p_amount
WHERE iban = p_to_iban;
-- 4. Запись транзакции (операционный платеж)
INSERT INTO payments_operational (from_iban, to_iban, amount, currency_id,
status, comment, payment_datetime)
VALUES (p_from_iban, p_to_iban, p_amount, p_currency_id, 'completed',
p_comment, NOW());
COMMIT;
END;

```

13. Добавьте недостающие комментарии в базу данных.

Рекомендуемые значения для ENUM полей:

```

-- employees.position
COMMENT ON COLUMN employees.position IS 'Должность сотрудника:
MANAGER, TELLER, CONSULTANT, DIRECTOR';

-- payments_archive.status и payments_operational.status
COMMENT ON COLUMN payments_archive.status IS 'Статус платежа:
PENDING, COMPLETED, FAILED, CANCELLED';
COMMENT ON COLUMN payments_operational.status IS 'Статус платежа:
PENDING, COMPLETED, FAILED, CANCELLED';

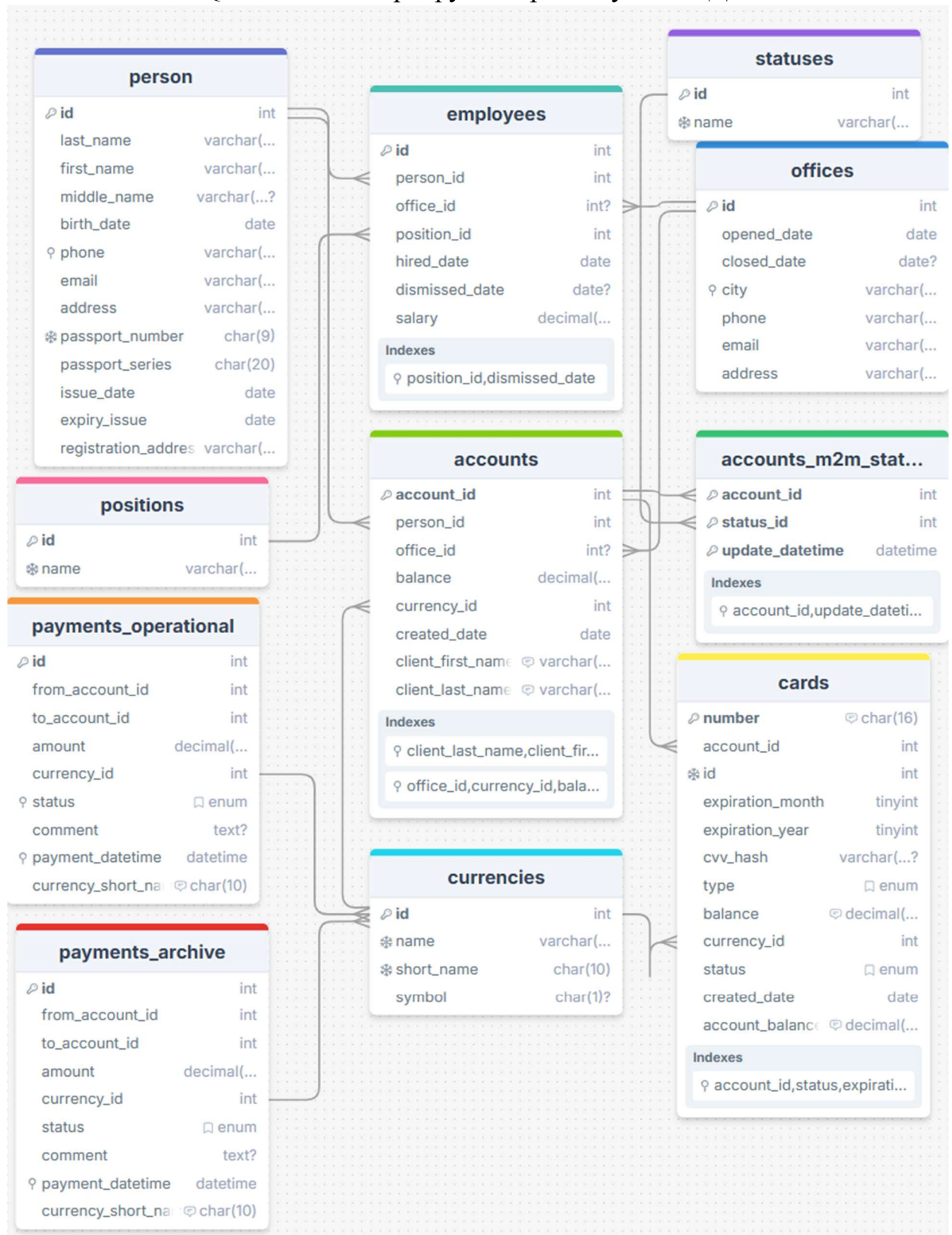
-- cards.type
COMMENT ON COLUMN cards.type IS 'Тип карты: DEBIT, CREDIT';

-- cards.status
COMMENT ON COLUMN cards.status IS 'Статус карты: ACTIVE, BLOCKED,
EXPIRED, LOST';

-- statuses.name
COMMENT ON COLUMN statuses.name IS 'Статус счета: ACTIVE, BLOCKED,
CLOSED, FROZEN';

```

14. Обновите логическую модель, экспортируйте окончательную версию модели в виде SQL-кода и импортируйте в реальную СУБД.



Исходная модель

1. Какие каскадные операции необходимы в этой базе данных?

owners → accounts:

ON DELETE RESTRICT

SET NULL

accounts → payments_operational:

ON DELETE CASCADE.

accounts → payments_archive:

ON DELETE CASCADE.

site_pages → site_pages:

ON DELETE CASCADE

ON UPDATE CASCADE

accounts → accounts_m2m_statuses:

ON DELETE CASCADE

2. Существует ли возможность аномалий операций вставки, обновления, удаления данных?

Таблица accounts (счета)

Аномалии вставки:

Нельзя создать счёт, если соответствующий владелец из people ещё не существует

Если person_id nullable — возникает логическая несостыковка (счёт без владельца)

Аномалии обновления:

Изменение person_id в people или iban в accounts без каскада приведёт к нарушению ссылочной целостности

Аномалии удаления:

Удаление владельца из people приведёт к "осиротевшим" счетам, которые не привязаны ни к какому владельцу

Таблицы payments_operational и payments_archive (платежи)

Аномалии вставки:

Нельзя записать платёж, если один из участвующих счетов (from_iban/to_iban) ещё не создан

Без внешних ключей возможны платежи, ссылающиеся на несуществующие счета

Аномалии обновления:

Изменение iban в accounts без каскада сделает ссылки в платежах невалидными

Аномалии удаления:

Удаление счёта приведёт либо к потере истории платежей, либо к невозможности удалить счёт (в зависимости от правил FK)

Проблемы дублирования:

Разделение на операционную и архивную таблицы требует атомарного переноса данных, иначе возможны потерянные или дублированные записи

Таблица statuses (статусы)

Аномалии вставки:

Возможны дублирующиеся названия статусов без уникального ограничения на name

Аномалии обновления:

Изменение id статуса без каскада нарушит связи в accounts_m2m_statuses

Аномалии удаления:

Удаление статуса без каскада создаст "висячие" ссылки в таблице связей

Таблица accounts_m2m_statuses (связь счетов и статусов)

Аномалии вставки:

Нельзя добавить связь статуса и счёта, если отсутствует соответствующая запись в accounts или statuses

Аномалии обновления:

Изменение iban или status_id без CASCADE приведёт к рассогласованию связей

Аномалии удаления:

Удаление счёта или статуса без каскада создаст "осиротевшие" записи в таблице связей

Таблица people (владельцы)

Аномалии обновления:

Изменение id владельца без каскада нарушит ссылки в accounts

Аномалии удаления:

Удаление владельца может разорвать связи со счетами

Таблица site_pages (страницы сайта)

Аномалии вставки:

Нельзя создать дочернюю страницу до создания родительской (если FK требует существования родителя)

Аномалии обновления:

Изменение `sp_parent` может создать циклические ссылки ($A \rightarrow B$, $B \rightarrow A$) без дополнительных проверок

Аномалии удаления:

Удаление родительской страницы без каскада оставит дочерние страницы с ссылками на несуществующий `id`

3. Можно ли использовать схемы «звезда» или «снежинка» с этой базой данных, чтобы избежать некоторых аномалий операций с данными?

Нет, текущая структура не подходит для прямого преобразования в схемы типа «звезда» или «снежинка», потому что это операционная (OLTP) база данных, а не аналитическая (OLAP). И применение «звезды» или «снежинки» не решит аномалии вставки/удаления/обновления — наоборот, чаще усложнит работу с транзакциями.

4. Составьте список всех функциональных зависимостей в базе данных

accounts:

`a_id` $\rightarrow$ `a_owner`;

`a_id` $\rightarrow$ `a_is_system`;

`a_id` $\rightarrow$ `a_balance`.

owners:

`o_id` $\rightarrow$ `o_name`.

payments_operational:

`po_id` $\rightarrow$ `po_from`;

`po_id` $\rightarrow$ `po_to`;

`po_id` $\rightarrow$ `po_money`;

`po_id` $\rightarrow$ `po_dt`

payments_archive:

`pa_id` $\rightarrow$ `pa_from`;

`pa_id` $\rightarrow$ `pa_to`;

`pa_id` $\rightarrow$ `pa_money`;

`pa_id` $\rightarrow$ `pa_dt`.

accounts_m2m_statuses:

`{ams_a_id, ams_s_id}` $\rightarrow$ `ams_last_update`.

statuses:

$s_id \rightarrow s_name.$

offices:

$of_id \rightarrow of_city;$

$of_id \rightarrow of_name;$

$of_id \rightarrow of_sales.$

site_pages:

$sp_id \rightarrow sp_parent;$

$sp_id \rightarrow sp_name.$

5. Существуют ли отношения, имеющие многозначные зависимости?

Нет отношений, имеющих многозначные зависимости.

6. Нарушает ли схема какие-либо «требования нормализации»?

owners:

Нарушение первой нормальной формы заключается в том, что атрибут `o_name` хранит в себе неатомарные данные объединяя имя фамилию и отчество в одном поле. Это нарушает принцип атомарности и создает практические трудности при работе с данными. Например, становится сложно выполнять сортировку по фамилии или выбирать записи только по имени поскольку все компоненты ФИО хранятся вместе в виде составной строки. Чтобы устранить эту проблему необходимо разделить составное поле `o_name` на три отдельных атомарных атрибута `last_name` для фамилии `first_name` для имени и `middle_name` для отчества. Такое преобразование обеспечит соответствие первой нормальной форме и упростит манипуляции с отдельными компонентами ФИО в запросах и прикладной логике.

7. Существуют ли какие-либо потенциальные проблемы с производительностью базы данных?

В данной базе данных могут возникать проблемы с производительностью связанные со сложной структурой связей между таблицами и необходимостью частого использования операций JOIN. Таблицы содержащие информацию о платежах особенно подвержены быстрому росту объема данных что в перспективе будет замедлять выполнение выборки и процессов архивирования. Операции обновления балансов счетов могут вызывать блокировки при одновременном выполнении множества транзакций. Таблица связывающая счета и их статусы также может достигать значительных размеров что скажется на скорости работы с ней. Иерархическая организация

страниц сайта может замедлять выполнение запросов из-за необходимости обработки древовидной структуры. Хранение суммы продаж в таблице офисов создает избыточность и требует постоянного пересчета при изменении исходных данных. Дополнительной проблемой является отсутствие индексов на полях которые являются внешними ключами что также негативно влияет на общую производительность системы.

8. Для каждого отношения в базе данных определите, в какой нормальной форме оно находится.

statuses: BCNF

owners: 0NF

offices: BCNF

payments_archive: BCNF

payments_operational: BCNF

accounts: BCNF

accounts_m2m_statuses: BCNF

site_pages: BCNF

9. Есть ли отношения с возможной, но ненужной дальнейшей нормализацией?

В таблице owners присутствует возможность нормализации через разделение поля name на три отдельных атрибута для имени фамилии и отчества однако возможно что архитектор базы данных сознательно отказался от этого решения считая его избыточным для текущих бизнес-процессов и требований к системе.

10. Можно ли добиться некоторого повышения производительности за счёт денормализации схемы?

В таблице accounts можно рассмотреть объединение со таблицей statuses перенеся поля статуса и даты создания взяв значение last_updated из accounts_m2m_statuses что ускорит получение информации о текущем статусе счета. Однако при таком подходе каждая смена статуса будет приводить к созданию новой записи аккаунта что вызовет дублирование данных для одного и того же счета при разных статусах. Также возрастает риск некорректности данных например из-за возможных опечаток при ручном вводе статусов. В результате такой денормализации таблица accounts будет соответствовать максимум второй нормальной форме.

Аналогично можно объединить таблицы owners и accounts перенеся поле имени владельца непосредственно в accounts что улучшит производительность при запросе информации о владельцах счетов. Но при

этом создание новых счетов для одного владельца будет приводить к многократному дублированию его данных в таблице accounts. Также сохраняется риск ошибок в данных например при неверном указании имени владельца для разных счетов. После такой денормализации таблица accounts также будет соответствовать только второй нормальной форме.

11. Можно ли добиться некоторого повышения производительности, добавив в схему кэширующие отношения?

Если бы между таблицами offices и owners существовала связь можно было бы добавить отдельную таблицу для хранения ежемесячных отчетов. В рамках этого подхода раз в месяц транзакции будут переноситься из transactions_operational в transactions_archive и параллельно будет вычисляться показатель sales для каждого офиса за прошедший период. Эти агрегированные данные будут сохраняться в специальной таблице отчетов. Такой подход оптимизирует получение статистики за длительные периоды например для получения годовых данных достаточно будет выбрать всего двенадцать записей по конкретному офису из таблицы отчетов и просуммировать значения вместо обработки всего массива транзакций за год.